

TITLE AND DATE

Document name: Project#1 Asynchronous Execution Lab Report

Document reference: kiddKid.cmpe311.fall26.project#1

Date of publication: September 16, 2026

LEAD ENGINEER: William Wang

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION: This document is the project document for Project#1 of the CMPE311 class. It contains customer, technical and testing requirements as well as the design and results of the validation testing.

DESCRIPTION: This design is to create an embedded system based upon an Arduino Uno R3 development platform that allows the user of the program to specify an LED number and its blink interval. The blinking of the LEDs must continue asynchronously without any additional input from the user. Included in this document are the customer requirements obtained from the customer, the high-level technical requirements derived from those customer requirements, the design, the testing scenarios and requirements, and the results of testing. Appended to this document is the code executed and a video of those tests that required video documentation.

RESULT SUMMARY: The project was a success, the embedded system design meets all testing and high-level requirements.

REFERENCES AND GLOSSARY

REFERENCES:

- ProjectAsyncTasking_draft – The definition of the project this document addresses
- CMPE310 Project #5 – A similar project assigned in CMPE310 in Spring 2025
- Arduino UNO R3 Product Reference Manual SKU A000066, 12/03/2024
- Async Programming in Arduino: Unleashing the Power of Non-Blocking Code, Mahdi Valizadeh, medium.com, 4/8/2024

DEFINITIONS:

“The User” – The person operating (not programming) the embedded system

“The System” – The embedded system being operated by The User

“The Customer” – The person(s) paying for the embedded system being designed and built

“The Developer” – The person(s) designing and building the System

“The Evaluator” – The person(s) that determine whether or not The System satisfies The Customer-requirements.

“The Customer-requirements” – The requirements defined by The Customer as satisfying The Contract.

“The Requirements” – The System’s high-level technical requirements derived from The Customer-requirements.

“The Educational-constraints” – Requirements imposed by the instructor unrelated to the embedded system that allow The System to be evaluated.

“The Company” – The organization The Customer has contracted with to build The System.

“The Contract” – The business document that legally binds The Company to provide some service or product to The Customer.

“serial-monitor” – The serial port used by the Arduino IDE to communicate with The User. “The

Reference-platform” – The configuration of The System used by The Developer to test and validate

The System. For this class, The System is the Arduino compatible ELEGOO Uno R3 development board.

ACRONYMS AND ABBREVIATIONS:

Arduino – an Italian open-source hardware and software company; also refers to a development board created by the company

arduino.h – header for a library of convenience functions specific to the Arduino development platform

AVR – A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology

ELEGOO – A Chinese company that develops and markets 3D printers and accessories

IDE – Integrated Development Environment

gcc – front end for the GNU Compiler Collection

Github – A widely used distributed SVC (Software Version Control) system

LED – Light Emitting Diode

REQUIREMENTS (Part I)

CONVENTIONS:

Must, shall or will – your design must satisfy the requirement

May – your design may satisfy the requirement but doesn't have to

Informative – the intent of the following description is to make the requirement more understandable

All customer requirements are started with "C.#".

All high-level technical requirements are started with "HL.#".

All testing/validation requirements are started with "T.#"

CUSTOMER REQUIREMENTS:

C1. The User will be able to choose the blink rate of the LED with a button on.

C2. The User will choose between 4 rates plus off.

C3. The blink rates must be set by an operator prior to the system being started

C4. Permitted blink rates must be perceivable by the viewer

C5. The minimum blink rate must be no slower than once every 5 seconds.

C6. The System must run upon an Arduino Uno R3 compatible board.

HIGH-LEVEL TECHNICAL REQUIREMENTS:

HL.1 The System must use at least 2 LEDs

HL.2 The System must use a standard Arduino compatible development board (e.g. the provided ELEGOO Uno R3)

HL.3 The System must communicate with The User via the Arduino IDE serial-monitor port

HL.4 Any use of the serial-monitor in HL.3 must be asynchronous and not affect the blinking of the LEDs.

HL.5 The User must set the blink interval of the LEDs in msec

HL.6 The blink rate of each LED must be constant unless changed by The User

REQUIREMENTS (Part II)

HIGH-LEVEL TECHNICAL REQUIREMENTS:

HL.1 The System must use at least 2 LEDs

HL.2 The System must use a standard Arduino compatible development board (e.g. the provided ELEGOO Uno R3)

HL.3 The System must communicate with The User via the Arduino IDE serial-monitor port

HL.4 Any use of the serial-monitor in HL.3 must be asynchronous and not affect the blinking of the LEDs.

HL.5 The User must set the blink interval of the LEDs in msec

HL.6 The blink rate of each LED must be constant unless changed by The User

EDUCATIONAL RESTRAINTS REQUIREMENTS:

EC.1 The Developer must use the version of gcc used by Arduino IDE 2.3.3 or later for the AVR ATmega328p to cross compile any program used by The System

EC.2 The Developer must have any software used in The System hosted on a github repository. This software does not need to include libraries, macros or other code not developed by The Developer.

EC.2.1 The Evaluator must be able to (a) download from the repository any software, documentation and other material needed for their evaluation, (b) cross compile any software, and (c) run that software on The Reference Platform.

EC.3 LEDs must be connected to Arduino digital Pins 2 & 3.

EC.4 The Evaluator must be able to successfully have The System perform the dialog in Table 5.

EC.5 The Developer must do bare metal programming, i.e. they cannot use any 3rd party multitasking libraries or programs such as FreeRTOS

DESIGN:

DESIGN PRE-REQUISITES:

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better

DEVELOPMENT PLATFORM:

1. See DESIGN PRE-REQUISITE 2 above

ANY ADDITIONAL DESIGN CONSIDERATIONS:

1. None

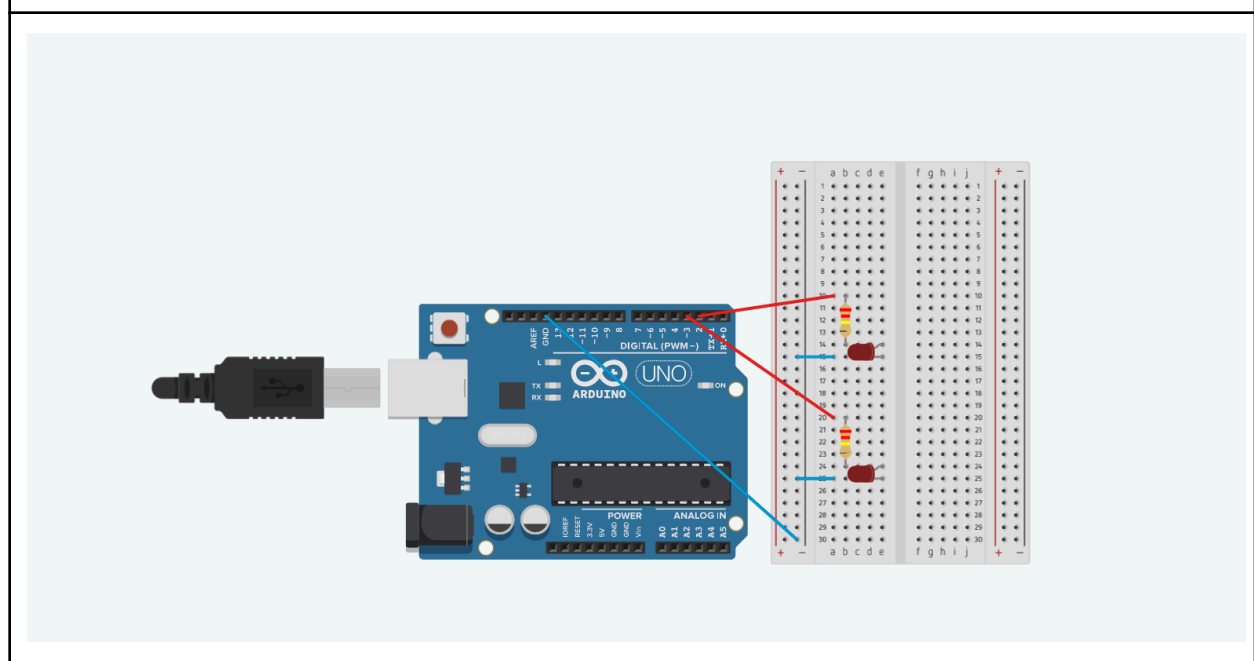
See Appendix A for the logical flow of the project program.

See Appendix B for the code executing on the Arduino Uno R3.

TESTING AND VALIDATION

DESCRIPTION: The tests that have to be performed and validated are shown in Table 2. These tests were performed on the testbed shown in Figure 1. Table 1 shows a dialog that must be successfully performed by the embedded system though testing must be more thorough than this. The results of testing are shown in Table 3. When necessary, a video of the test is provided along with this report.

Figure 1. Testbed Setup. LEDs connected to digital pins 2 & 3.



TESTING PLATFORM:

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better
3. Power: Testing platform powered through USB cable, LEDs connected directly to Digital Outputs

Table 1. Required Test Dialog (*blue* are the user inputs).

Serial port I/O	Notes
What LED? (1 or 2) <i>2</i>	
What interval (in msec)? <i>600</i>	LED2 starts blinking at an interval of 300ms on and 30,0ms off. LED1 is unaffected.

What LED? (1 or 2) 1	
What interval (in msec)? 1600	LED1 starts blinking at an interval of 8800ms on and 800ms off. LED2 is unaffected.

TESTING AND VALIDATION REQUIREMENTS:

Table 2. Testing and Validation Requirements

T.0 All testing must be done on the Arduino Uno R3-compatible testbed with LEDs connected to pins 2 and 3.
T.1 The Serial Monitor dialog must allow The User to select an LED and enter its blink interval.
T.1A The blink interval of one LED must be able to be changed without affecting the other LED.
T.2 The blink interval of an LED must not change until the new input is complete.
T.3 The User must enter the blink interval in milliseconds.
T.4 The interval entered by The User must be correctly reflected by the selected LED.
T.4.1 The non-selected LED must continue blinking at its previous interval.

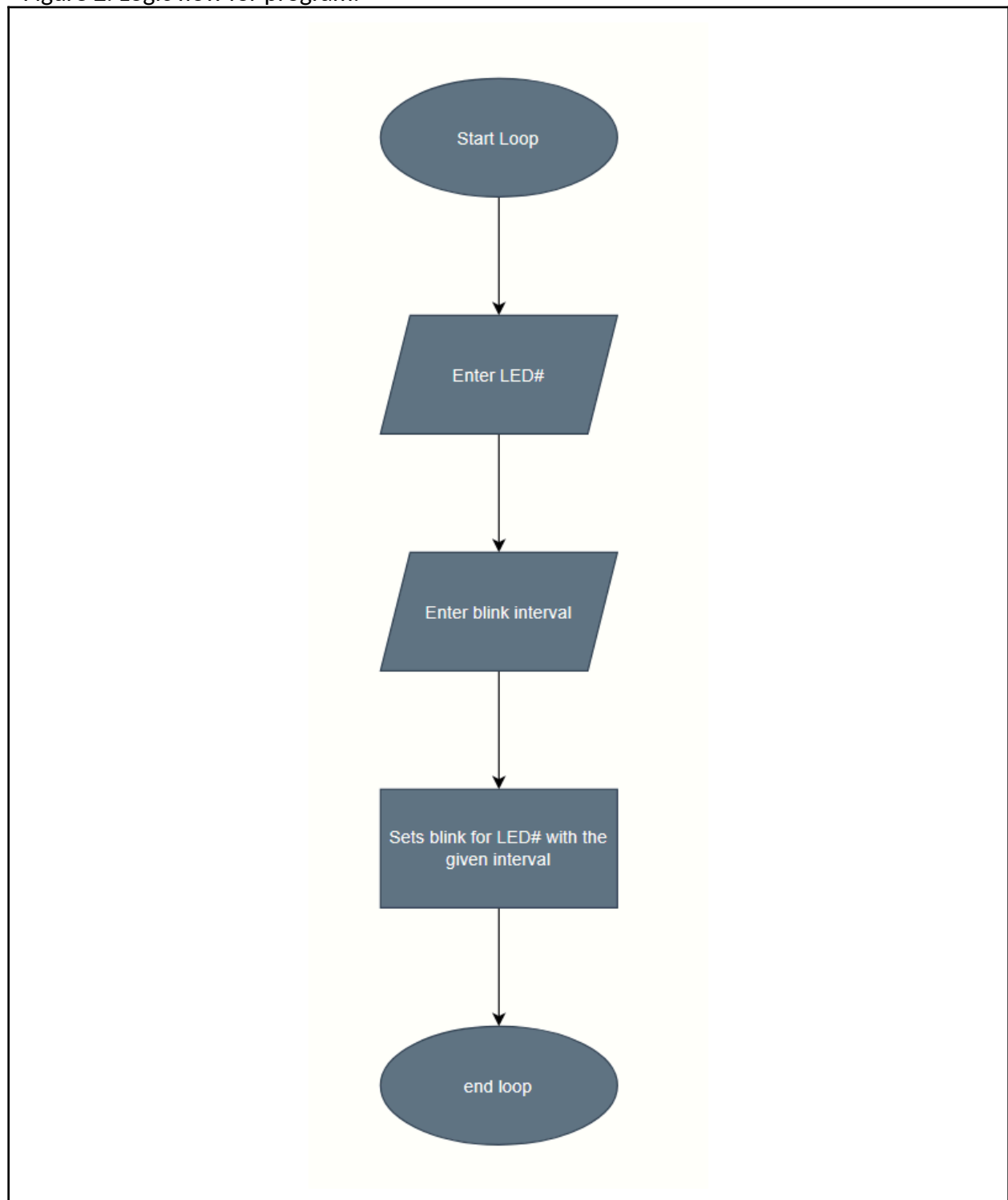
TESTING RESULTS:

Table 3. Results of tests

Test performed	Results
T.0	Satisfied. See video of test.
T.1	Satisfied. See video of test.
T.1.A	Satisfied. See video of test.
T.2	Satisfied. See video of test.
T.3	Satisfied. See video of test.
T.4	Satisfied. See video of test.
T.4.1	Satisfied. See video of test.

Appendix A. Logic Flow Chart

Figure 2. Logic flow for program.



APPENDIX B. DESIGN CODE

```
#include <Arduino.h>

int selectedLED = -1;

unsigned long blinkRateOne;
unsigned long blinkRateTwo;

unsigned long blinkTrackerOne;
unsigned long blinkTrackerTwo;

void setup() {
    // put your setup code here, to run once:
    Serial.begin(9600);

    pinMode(2, OUTPUT);
    pinMode(3, OUTPUT);

    Serial.println("What LED? (1 or 2):");
}

void loop() {
    // put your main code here, to run repeatedly:

    // Tracks the time
    unsigned long currentTime = millis();

    // Checks the time to see if LED1 should blink
    if (blinkRateOne && (currentTime - blinkTrackerOne) >= (blinkRateOne
/ 2)){
        digitalWrite(2, !digitalRead(2));
        blinkTrackerOne = currentTime;
    }
}
```

```

// Checks the time to see if LED2 should blink
if (blinkRateTwo && (currentTime - blinkTrackerTwo) >= (blinkRateTwo
/ 2)){
    digitalWrite(3, !digitalRead(3));
    blinkTrackerTwo = currentTime;
}

if ((Serial.available() > 0) && (selectedLED == -1)){
    selectedLED = Serial.parseInt();

    while (Serial.available() > 0) {
        Serial.read();
    }

    Serial.println("What interval (in msec):");
}

else if ((Serial.available() > 0) && (selectedLED != -1)){

    // Sets new blink rate based on LED chosen before
    if (selectedLED == 1){
        blinkRateOne = Serial.parseInt();
    } else {
        blinkRateTwo = Serial.parseInt();
    }

    selectedLED = -1;
    while (Serial.available() > 0) {
        Serial.read();
    }
    Serial.println("What LED? (1 or 2):");

}

}

```

END OF DOCUMENT